LumenAI

A Memory-Bounded, Multi-Model System for
Prompt-Driven Image Editing and Generation

Technical Report

AI/ML Team 06

October 2026

Abstract. LumenAI is a photo editor and image generator that is driven by plain text. A user types what they want, for example “make the backpack navy blue” or “remove the person on the left”, and the system routes the request to the pipeline that does that job best. The system combines Qwen-Image 2.1 with its Qwen3-VL text encoder, SAM 2.1, FLUX.2 klein 4B, Depth Anything V2, TADSR, IC-Light, Magenta and SPAR3D, and every model runs within a 16 GB GPU memory budget. This report describes the architecture, the request guard, the routing logic, each editing pipeline with its mathematics, the systems work that keeps memory and latency low, and a benchmark of 10 task families against the alternatives we evaluated. Its central technical contribution is a training-free, part-level recolouring method with an interactive colour slider: a soft region is built by an algebra of masks, the colour is moved in the perceptual OKLab space while the original shading is kept, and every pixel outside the selected part is returned bit-identical by construction. On 48 recolouring tasks it shows zero colour leakage, a 0.6 degree hue error and a shading correlation of 0.997, while a generative editor reaches 3.69 ΔE_{00} leakage and 0.735 shading correlation.

Contents

1 Introduction **2**

2 System architecture **2**
2.1 Request lifecycle . . . 2
2.2 Routing . . . 3

3 Models and model tiers **3**

4 Editing pipelines **5**
4.1 Selection . . . 5
4.2 Local edits and frequency-aware blending . . . 5
4.3 Smart object removal . . . 6

5 The recolour slider **6**
5.1 From words to a program . . . 7
5.2 The object frame . . . 7
5.3 Soft masks with a hard guarantee . . . 7
5.4 Colour transfer in OKLab . . . 9
5.5 The slider . . . 10
5.6 Examples and results . . . 10

6 Further pipelines **11**
6.1 Pose transfer through words . . . 11
6.2 Relighting with light transfer . . . 11
6.3 Depth-aware portrait bokeh . . . 12
6.4 Dolly video from one photo . . . 13
6.5 Glow light . . . 13
6.6 Auto collage . . . 15
6.7 Other tools . . . 15
6.8 The final pipeline at a glance . . . 16

7 Systems engineering **17**

8 Benchmarks **18**
8.1 Speed and memory . . . 19
8.2 Quality . . . 20

9 Limitations **21**

10 Conclusion **22**

A API endpoints **22**

B Reproducibility **22**

1 Introduction

Professional image editors are powerful but slow to learn. Generative models make many edits possible from a sentence, but a single large model is a poor fit for all of them. It redraws pixels that should stay untouched, it changes colours in shadows, and it needs more GPU memory than a typical 16 GB card offers. LumenAI takes a different route. It treats the language model as a dispatcher and as a source of understanding, and it gives each kind of edit a specialised pipeline: some are generative, some are exact mathematics, and some are a mix of both.

Goals.

(i) **One text box for everything.** Generation and editing from natural language, with optional click, box or brush selections.

(ii) **A 16 GB budget.** Every model must fit in 16 GB of GPU memory at its peak.

(iii) **Edits that respect the photo.** Pixels outside the intended edit stay identical wherever the method allows it.

(iv) **Measured decisions.** Every model choice is backed by a benchmark (Section 8).

Contributions.

- A production architecture with one shared engine and four side containers, all within 16 GB per model (Section 2).
- A model picker with three tiers, Lumen 1.1 Lumina, Lumen 1.2 Kroma and Lumen 1.3 Lux, which trade speed for detail (Section 3).
- The **recolour slider**: mask algebra plus OKLab colour transfer with exact protection outside the part, and an interactive slider that re-renders on the CPU in about a second (Section 5).
- Training-free pipelines for removal, pose transfer, relighting, depth-aware bokeh and dolly video (Section 4).
- Systems work: 4-bit quantisation, encoder offloading, prompt-embedding caching and seed mixing, which together keep the main pipeline at 14 GB and fixed a real failure mode (Section 7).

2 System architecture

Figure 1 shows the system. The browser holds a static web application (a landing page and the editor). It talks to a FastAPI service that runs on a serverless GPU container. Images travel as data URLs in JSON. The main container holds one *engine*: Qwen-Image 2.1 in 4-bit nf4, whose text encoder is a full Qwen3-VL vision-language model, plus SAM 2.1, Depth Anything V2 Small, an OpenPose detector, a prompt-injection classifier and the watermark encoder. Five models live in their own containers because of their size or their dependencies: FLUX.2 klein 4B (Lumen 1.1 Lumina, also used for product angles), the TADSR upscaler, which pins an older PyTorch, IC-Light (a Stable Diffusion 1.5 relighting model), Magenta style transfer (TensorFlow) and the 3-D models SPAR3D and Stable Fast 3D (SF3D) for View in 3D. A shared volume caches all weights, and idle containers scale to zero.

2.1 Request lifecycle

Every request follows the path in Fig. 2. The request guard runs first on any free text. The router then chooses a pipeline. Large uploads are processed at a working size of at most 2048 px on the long side; the result is scaled back and, for local edits, only the edited region of the full-resolution photo is replaced. Each response carries the execution time and the peak GPU memory of the request, which the editor shows in a “Last run” card.

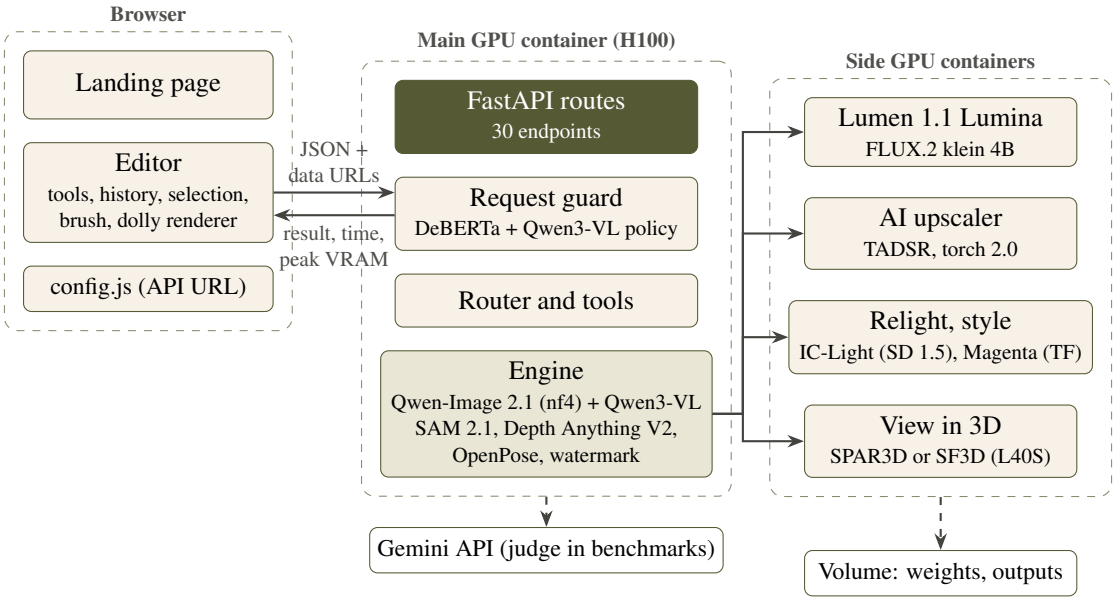


Figure 1: System architecture. One shared engine serves almost every tool; models that would break the 16 GB budget or need other libraries run in their own containers and are called remotely.

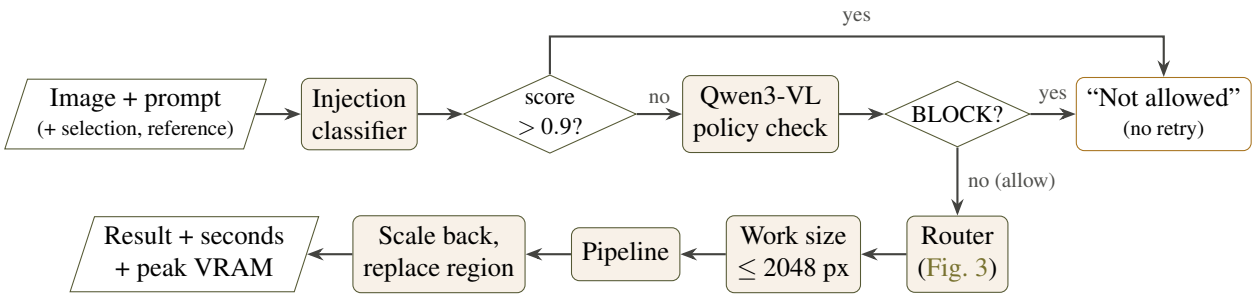


Figure 2: Life of a request. The guard blocks prompt injections with a classifier and policy violations with a question to Qwen3-VL; answers are cached, and the check fails open on internal errors so a fault never blocks a normal edit.

2.2 Routing

The router reads the words of the request and the presence of a selection or a reference photo (Fig. 3). A reference photo with pose words ("pose like this", "sitting like") goes to the pose pipeline; with style words it goes to style transfer. Removal verbs go to smart removal. A colour word together with a change verb, in a short request, goes to the exact recolour pipeline. Blur words go to deblurring. Everything else becomes an instruction edit, local if a selection exists and global otherwise. When a specialised route cannot proceed (no person in the pose photo, nothing found to remove), it either explains why or falls back to the instruction edit.

3 Models and model tiers

Table 1 lists the models in use. Specialist tools always use their best pipeline. For new images and for instruction edits, the user picks one of three tiers in the prompt bar (Fig. 4).

Table 2 compares the three tiers in detail. The differences come from three choices: the diffusion model (a small step-distilled model against a large one), the number of denoising steps, and whether Qwen3-VL first rewrites the request.

Why the wait differs from the model time. Every request first passes the request guard (about 2 s, cached per prompt) and sends the photo as a data URL. Lumina lives in its own container, which shuts down after 10 idle minutes; the next request then loads 8 GB of weights and runs a warm-up image before it answers, which

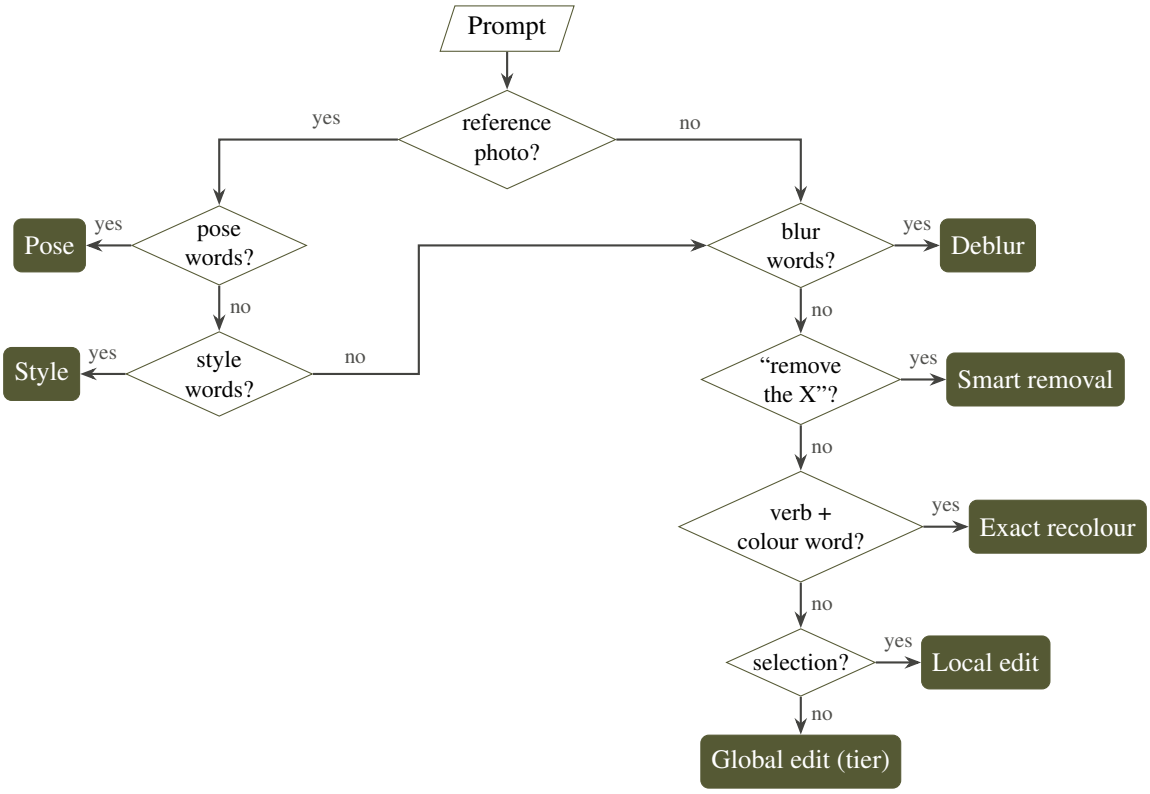


Figure 3: The prompt router. Filled boxes are pipelines (Section 4); only the global edit depends on the chosen model tier.

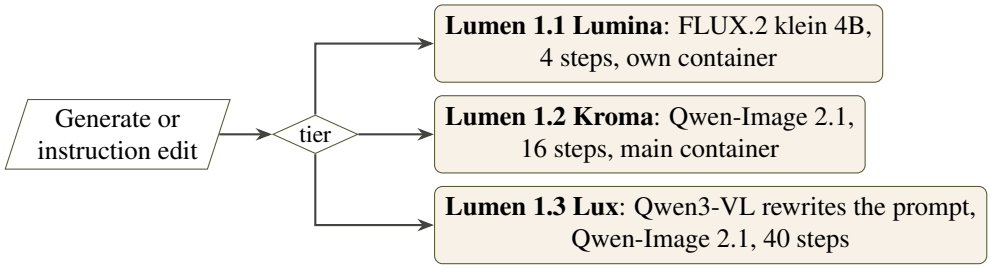


Figure 4: The three model tiers. "Another version" keeps the tier and increments the seed; "Redo with Lux" keeps the seed and switches to the best tier.

is why a single measured Lumina request took 40 s although the model itself needs about a second. Kroma and Lux share the main container, which stays warm while the editor is in use; Lux adds the prompt rewrite and 2.5 times the steps. The editor therefore shows the measured time and peak memory of each request instead of a promised time.

Prompt rewriting in Lux. Qwen3-VL turns a short request into a detailed description (60 to 110 words for generation, at most 60 words for edits, written while looking at the photo) that keeps every object, colour, count and quoted text the user gave. In our text-to-image benchmark this raised the prompt-following score from 8.4 to 9.7 out of 10.

Seed mixing. The noise seed actually used is

$$s' = \mathrm{SHA256}\big(s \,\|\, \kappa(x)\big) \bmod 2^{31}, \tag{1}$$

where s is the user-facing seed and $\kappa(x)$ is a key of the input: the prompt for generation, or a 32×32 thumbnail of the image for edits. Without this, "generate" at seed 7 followed by "edit" at seed 7 started the edit from the exact noise that had produced the picture, and the model slid back to the same image, ignoring requests such

Table 1: Models used by LumenAI and what they do. Peak memory is measured on the tool that uses the model most.

Model	Role in LumenAI	Peak GPU
Qwen-Image 2.1 (nf4)	generation, instruction edits, removal, outpainting (LoRA), add object, relight, product shots	14.2 GB
Qwen3-VL (Qwen-Image's text encoder)	grounding boxes, photo-specific suggestions, pose description, prompt rewriting, safety policy	shared
FLUX.2 klein 4B	Lumen 1.1 Lumina: fast generation and edits (4 steps), own container	13.4 GB
SAM 2.1	selection from clicks or boxes, masks for removal and recolour	<1 GB
Depth Anything V2 Small	relative depth for bokeh and dolly video	<1 GB
OpenPose (controlnet-aux)	body joints for pose transfer	<1 GB
DeBERTa-v3 injection classifier	request guard, prompt-injection detection (CPU)	0.8 GB
TADSR	one-step diffusion super-resolution, own container	5.7 GB
SPAR3D	View in 3D (default): point cloud first, then mesh; fuller depth and back, own container (L4)	11.0 GB
Stable Fast 3D (SF3D)	View in 3D (fast option): textured mesh in one pass, own container (T4)	11.0 GB
Magenta (TensorFlow Hub)	Style, fast engine: feed-forward arbitrary style transfer, own container (T4)	1.3 GB
IC-Light (SD 1.5, fc)	diffusion relighting with a text-described light, own container	≈4.5 GB[†]
CLIP ViT-B/32	auto collage: photo ranking, prompt relevance, near-duplicate removal	<1 GB
Gemini (API)	second judge in the quality benchmark (not used by the editor)	none

[†] Estimated from the model size; the editor shows the measured value of every request.

as "add a woman sitting on the sofa". With mixing, results stay reproducible (same seed and same input give the same output) but an edit never reuses its input's own noise.

4 Editing pipelines

4.1 Selection

A click gives SAM 2.1 positive (or, with Alt, negative) points. A dragged box restricts SAM to the box, and any mask pixel more than 2% of the image size outside the box is discarded. The brush paints a mask directly in the browser; with "Snap to the object" the stroke is passed to SAM 2.1 as a prompt, so the mask follows the object's outline. All three produce the same binary mask $M \in \{0,1\}^{H\times W}$, which every tool accepts.

4.2 Local edits and frequency-aware blending

For an edit inside a mask M, the model only sees a crop window around M with a margin of $0.8\times$ the region's longer side (at least 384 px), at 768 px resolution. The result G is blended back into the original O with a Laplacian pyramid. With pyramids $\mathscr{L}_k(\cdot)$ for $k = 0,\dots,n-1$ ($n = 6$) and a band width per level that grows geometrically from fine to coarse,

$$b_k = b_{\text{high}} \left(\frac{b_{\text{low}}}{b_{\text{high}}} \right)^{k/(n-1)}, \qquad b_{\text{high}} = 3 \text{ px}, \; b_{\text{low}} = 48 \text{ px}, \tag{2}$$

the weight of level k is a linear ramp of the inner distance $D(p)$ to the region border, $w_k(p) = \text{clip}(D(p)/b_k, 0, 1)$, downsampled to level k, and

$$\mathscr{L}_k(\text{out}) = w_k\, \mathscr{L}_k(G) + (1 - w_k)\, \mathscr{L}_k(Y), \qquad Y = M \odot G + (1 - M) \odot O. \tag{3}$$

Fine detail therefore switches over 3 px (edges stay sharp) while colour and light blend over 48 px (no visible seam). After collapsing the pyramid, every pixel outside the dilated region is copied from O, so it is

Table 2: The three model tiers compared. Model times are warm GPU times at 1024 px from the benchmark (Section 8); the last time row is the full wait measured in the editor, which adds the request guard, image transfer and, for Lumina, the start of its own container.

	Lumen 1.1 Lumina	**Lumen 1.2 Kroma**	**Lumen 1.3 Lux**
Diffusion model	FLUX.2 klein 4B, step-distilled rectified-flow transformer	Qwen-Image 2.1, MMDiT transformer, nf4	Qwen-Image 2.1, MMDiT transformer, nf4
Text encoder	Qwen3 (8 GB), nf4, on the GPU	Qwen3-VL, nf4, kept on the CPU between prompts	same as Kroma
Denoising steps	4	16	40
Prompt	as typed	as typed	rewritten by Qwen3-VL (60 to 110 words for images, up to 60 for edits, written while looking at the photo)
Runs in	its own GPU container	the main API container	the main API container
Peak GPU memory	13.35 GB	12.5 GB (image), 13.8 GB (edit)	12.5 GB (image), 13.8 GB (edit)
Model time, new image	1.1 s	about 0.4× Lux (estimated from 16 of 40 steps)	20.6 s
Model time, edit	1.7 s	about 0.4× Lux (estimated)	35 s
Full wait in the editor (2026-10-07)	40 s when its container was asleep; a few seconds when warm	20 s	30 s
Text to image (judge, 0 to 10)	prompt 9.7, quality 9.7	not benchmarked	prompt 9.7, quality 9.8
Instruction edits (GEdit, overall)	**7.80**	7.36 (Qwen-Image)	7.36 (Qwen-Image)
Keeps logos and text	weaker (DINOv2 0.815)	strong	strong (DINOv2 0.900)
Best for	drafts, quick ideas, many variations	everyday edits and images	final images, long or detailed requests
Weak points	cold start of its own container; redraws small text	less fine detail than Lux	slowest; the rewrite adds a few seconds

bit-identical.

4.3 Smart object removal

Removal (Fig. 5) combines generation with checks. Qwen3-VL reads what the selection contains; Qwen-Image draws a 512 px draft of the scene without it and refines it at 768 px; Qwen3-VL then checks that the object is gone. If not, up to three fallbacks run (a firmer instruction, a larger mask, a different seed). The object's shadow is found by letting the generative model show it: the difference between the photo and the model's own removal, restricted to a band around the object, marks the shadow, which is then removed too. The final composite uses the blend of Section 4.2. On the RORD benchmark this pipeline reaches an LPIPS of 0.108 inside the hole, against 0.151 for LaMa and 0.179 for MAT (Section 8).

5 The recolour slider

Changing the colour of an object, or of part of an object, is one of the most frequent edits. Generative editors do it poorly: they redraw the object, flatten its shading, drift the background and often change the wrong part. We treat recolouring as an exact image-processing problem instead. A language model decides *where* (which part), and closed-form colour science decides *how*. Because no diffusion runs, the colour step takes about a second on a CPU, which makes an interactive slider possible. Figure 7 shows the full pipeline.

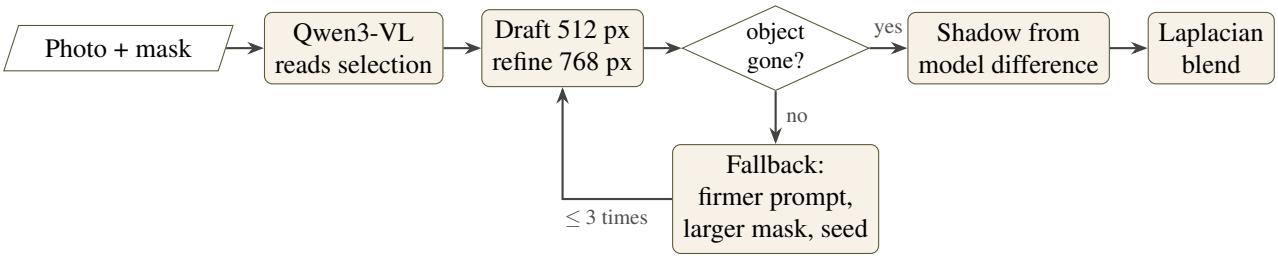


Figure 5: Smart removal: generation, self-check, fallbacks and shadow removal.

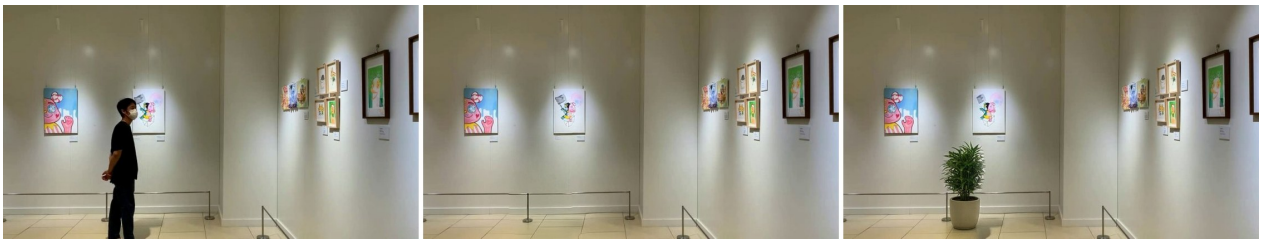


Figure 6: Removal example from the product. Left: a gallery photo with a visitor. Middle: the visitor and the shadow removed. Right: the same spot filled with "a potted plant" using the add-object tool.

5.1 From words to a program

Qwen3-VL turns the request into a small JSON program, for example

```
{"object": "bag", "target_color": "red", "region": {"op": "minus", "args": [
{"op": "half", "side": "lower", "frac": 0.333}, {"op": "part", "name": "pocket"}]}}
```

for "make the bottom third red except the pocket". The grammar has leaves `object`, `half`, `band`, `stripes`, `border` and `part` (a named sub-part found by a Qwen3-VL box and SAM), and the operators `and`, `or`, `minus` and `not`. Programs are validated; an invalid reply falls back to a rule-based parser that covers the common phrasings. If the program names a different object than the one that is masked ("shirt" in a photo of a person), that name is turned into a part of the masked object; without this step the whole person was recoloured.

5.2 The object frame

Let $\Omega = \{p : M(p) = 1\}$ be the object pixels and $N = |\Omega|$. The scale of the object is $\sigma_M = \sqrt{N}$ and the seam width is

$$\tau = \max(0.75,\ \varsigma\,\sigma_M), \qquad \varsigma = 0.006. \tag{4}$$

Directions ("upper", "left") are taken in the object's own frame: the principal axes $\mathbf{e}_1, \mathbf{e}_2$ of the centred pixel coordinates (PCA). If the major axis lies within 20° of an image axis, the image axes are used instead, so that a mug's stripes stay horizontal even when a handle tilts its shape. A side σ maps to the unit vector $\mathbf{u}_\sigma \in \{\pm\mathbf{e}_1, \pm\mathbf{e}_2\}$ closest to the image direction, and every pixel gets the coordinate $t(p) = \mathbf{u}_\sigma \cdot p$.

5.3 Soft masks with a hard guarantee

All cuts use the compact smoothstep (Fig. 8)

$$S(x) = v^2(3 - 2v), \qquad v = \mathrm{clip}\left(\frac{x+2}{4}, 0, 1\right), \tag{5}$$

which is exactly 0 for $x \le -2$ and exactly 1 for $x \ge 2$. A signed distance divided by τ therefore produces a seam of width 4τ with no tail: nothing changes further than 2τ from a cut.

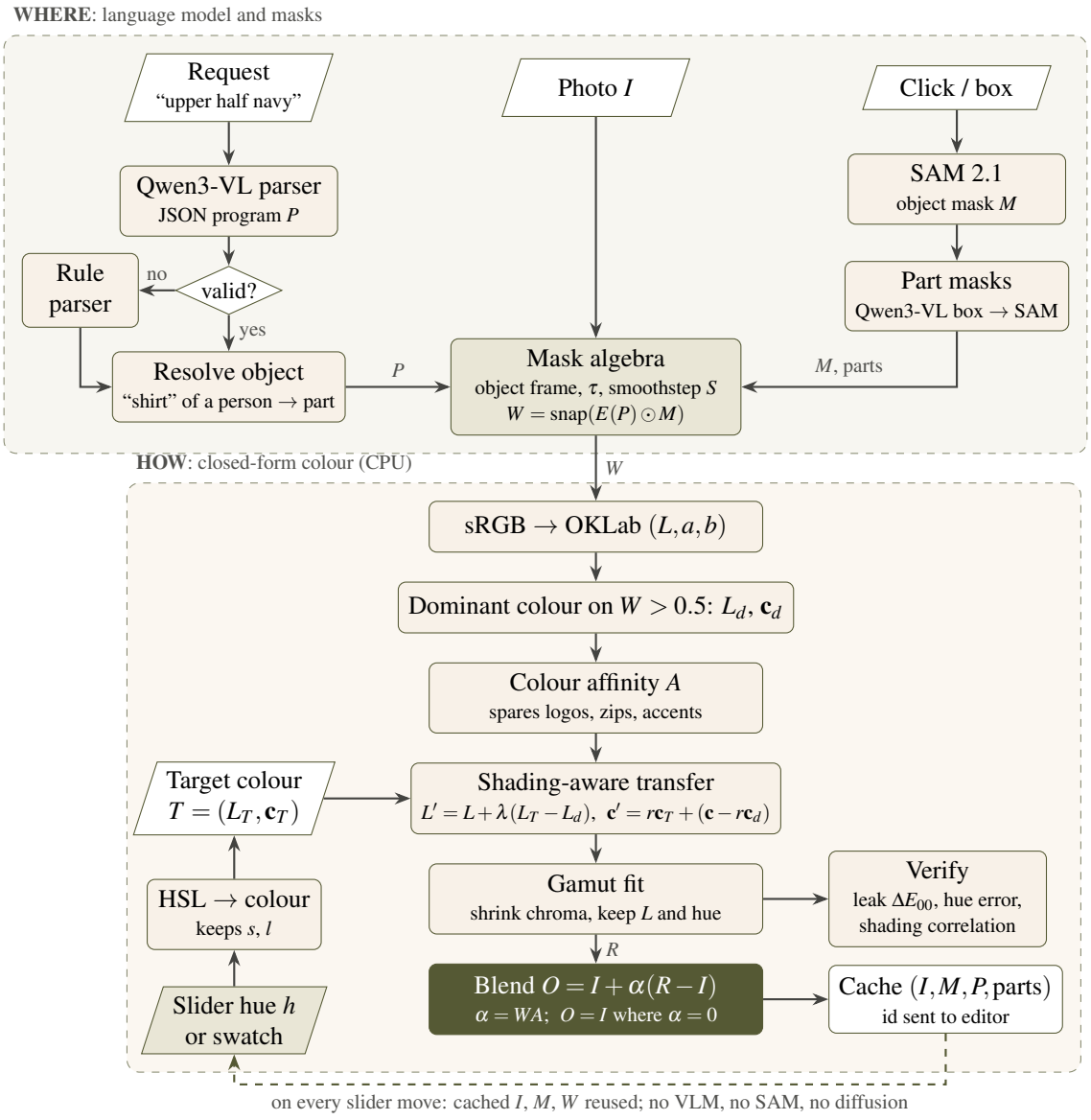


Figure 7: The recolour pipeline. The upper part decides which pixels belong to the requested part; the lower part moves their colour. The dashed loop is the slider: it re-runs only the lower part with a new target colour.

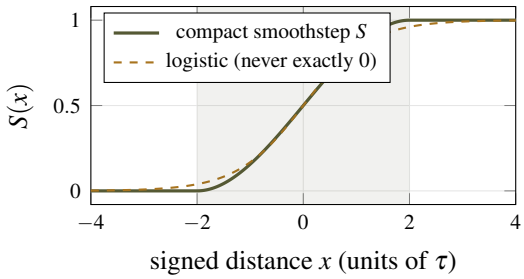


Figure 8: The seam function. Unlike a logistic curve, S reaches 0 and 1 exactly at $\pm 2\tau$, which is what makes the protection guarantee exact.

Leaves.

- **half**(σ, f): the cut is an *area quantile*, not the bounding-box midline. With $c = Q_{1-f}\{t(p) : p \in \Omega\}$, the part holding a fraction f of the object's area is $E = S\big((t-c)/\tau\big)$. On a tapered bag "upper half" then really contains half of the bag.
- **band**(σ, f_0, f_1): $E = S\big((c_0 - t)/\tau\big)\, S\big((t-c_1)/\tau\big)$ with $c_i = Q_{1-f_i}$.
- **stripes**(n, ω): with t spanning $[t_{\min}, t_{\max}]$ on the object, cycle length $\ell = (t_{\max} - t_{\min})/n$, coloured length $o = \omega\ell$ ($\omega = 0.5$ by default), phase $q = (t - t_{\min} - (\ell - o)/2) \bmod \ell$ and exact signed distance

$$d = \begin{cases} \min(q,\, o-q) & q < o \\ -\min(q-o,\, \ell-q) & \text{otherwise,} \end{cases} \qquad E = S\left(\frac{d}{\max(0.5, \min(\tau, o/6))}\right). \tag{6}$$

  The edge width is capped at a third of a stripe so that thin stripes stay crisp.
- **border**(w): with the distance transform $D(p)$ to the outside of Ω, $E = S\big((w\sigma_M - D)/\tau\big)$.
- **part**: the part mask, smoothed by a Gaussian of width τ.

Operators. On soft masks the algebra is

$$E_{a\wedge b} = E_a E_b, \qquad E_{a\vee b} = E_a + E_b - E_a E_b, \qquad E_{a\setminus b} = E_a(1-E_b), \qquad E_{\neg a} = 1 - E_a. \tag{7}$$

On hard masks these reduce to the usual set operations. The region weight is finally intersected with the object and snapped:

$$W = \mathrm{snap}_\varepsilon\big(E \cdot M\big), \qquad \mathrm{snap}_\varepsilon(x) = \begin{cases} 0 & x < \varepsilon \\ 1 & x > 1-\varepsilon \\ x & \text{otherwise,} \end{cases} \qquad \varepsilon = \tfrac{1}{510}. \tag{8}$$

The value ε is half of one 8-bit level, so snapping never changes a visible value.

5.4 Colour transfer in OKLab

Colour space. OKLab is perceptually uniform: equal distances look like equal colour differences, and lightness L is separated from the chroma plane $\mathbf{c} = (a, b)$. With γ^{-1} the sRGB decoding to linear light and fixed matrices $\mathbf{M}_1, \mathbf{M}_2$,

$$(L, a, b)^\top = \mathbf{M}_2 \sqrt[3]{\mathbf{M}_1\, \gamma^{-1}(I/255)}. \tag{9}$$

Dominant colour. On the core $\mathscr{C} = \{p : W(p) > 0.5\}$ we take medians, which ignore logos and zips: $L_d = \mathrm{med}_{\mathscr{C}}\, L$ and $\mathbf{c}_d = \mathrm{med}_{\mathscr{C}}\, \mathbf{c}$.

Colour affinity. Accents of a different colour (a black zip on a yellow bag) should not be repainted. With $\delta(p) = \|\mathbf{c}(p) - \mathbf{c}_d\|$, its median $m = \mathrm{med}_{\mathscr{C}}\,\delta$ and $\sigma_A = \max(0.035, 2.5m)$,

$$A(p) = \exp\left(-\frac{1}{2}\left(\frac{\max(\delta(p) - \sigma_A/2, 0)}{\sigma_A}\right)^2\right). \tag{10}$$

Pixels within $\sigma_A/2$ of the dominant colour get $A = 1$; clearly different colours fade out smoothly.

Shading-aware transfer. Let the target colour be $T = (L_T, \mathbf{c}_T)$. A naive replacement $\mathbf{c}' = \mathbf{c}_T$ would paint shadows as saturated as highlights and lose the fabric's texture. We use a shading ratio $r(p) = \mathrm{clip}\big(L(p)/L_d, 0, 1.6\big)$ and set

$$\mathbf{c}'(p) = r(p)\,\mathbf{c}_T + \big(\mathbf{c}(p) - r(p)\,\mathbf{c}_d\big), \tag{11}$$

$$L'(p) = \mathrm{clip}\big(L(p) + \lambda\,(L_T - L_d), 0, 1\big), \qquad \lambda = 0.7. \tag{12}$$

The first term scales the target chroma with the local shading, as real colour does in shadow; the second keeps each pixel's own deviation from the dominant colour, which is its texture. The lightness is shifted by a constant, so $\mathrm{corr}(L, L') = 1$ wherever no clipping occurs: shading is preserved exactly. The factor λ moves the overall brightness towards the target (a dark navy on a light yellow bag) without erasing the shading.

Gamut fit. Some $(L', \mathbf{c}')$ fall outside sRGB. Clipping RGB channels would shift the hue, so instead, for each such pixel, a bisection (10 steps) finds the largest $k \in [0, 1]$ such that $(L', k\,\mathbf{c}')$ is inside the gamut. Lightness and hue angle are kept, only chroma is reduced. Converting back gives the recoloured image R.

Blend. With $\alpha = W \cdot A$ (zeroed below ε),

$$O(p) = \begin{cases} \mathrm{round}\big(I(p) + \alpha(p)\,(R(p) - I(p))\big) & \alpha(p) > 0, \\ I(p) & \alpha(p) = 0. \end{cases} \tag{13}$$

Guarantee. Every pixel with $W(p) = 0$ satisfies $O(p) = I(p)$ bit for bit. This includes every pixel outside the object mask and every pixel more than 2τ outside a cut, because S is exactly zero there and snapping removes round-off. The pipeline verifies this after every call, together with the CIEDE2000 leak on protected pixels, the hue error of the edited part and the correlation of lightness before and after.

5.5 The slider

The first recolour stores (I, M, P, parts) in a small server cache (8 entries, least recently used) and returns an identifier. In the editor, picking a swatch or dragging the shade slider produces a new target colour: the slider value is a hue $h \in [0, 360)$, and the colour keeps the saturation and lightness of the current colour, clamped to $s \geq 0.35$ and $0.25 \leq l \leq 0.7$ so that every position is a usable colour. Each slider release sends only $(\text{id}, \text{colour})$. The server re-runs the lower half of Fig. 7 with the cached mask and program, without Qwen3-VL, SAM or diffusion, and replaces the current history step. On one CPU core the colour step takes 0.8 s at 0.5 megapixels and 2.5 s at 1 megapixel; the full first recolour, including the VLM program and SAM, takes about 3 s.

5.6 Examples and results

Figures 9 to 11 show the method on a real product photo. The slider sweep keeps the stitching, folds and the dark zip in every colour. The programs show that parts follow the object's area and shape. The internal maps show that the colour affinity spares the zip and the edges, and that the change outside the object is exactly zero. For the navy result the verification reports a maximal leak of 0.0 ΔE_{00}, a hue error of 0.5 degrees and a shading correlation of 0.99.

Algorithm 1: Recolour and slider re-render

Input: photo I, object mask M, program P, part masks, target colour T
Output: recoloured photo O and checks

$\tau \leftarrow \max(0.75, 0.006\sqrt{|M|})$; compute the object frame (PCA, snapped to image axes within 20°);
$E \leftarrow \textsc{Evaluate}(P.\text{region})$ using S, area quantiles and the soft algebra;
$W \leftarrow \text{snap}_{1/510}(E \cdot M)$; **if** $|\{W > 0.5\}| < 10$ **then fail** "part is empty";
$(L, \mathbf{c}) \leftarrow \text{OKLab}(I)$; $L_d, \mathbf{c}_d \leftarrow$ medians on $\{W > 0.5\}$;
$A \leftarrow$ colour affinity around $\mathbf{c}_d$;
$r \leftarrow \text{clip}(L/L_d, 0, 1.6)$; $\mathbf{c}' \leftarrow r\mathbf{c}_T + (\mathbf{c} - r\mathbf{c}_d)$; $L' \leftarrow \text{clip}(L + 0.7(L_T - L_d), 0, 1)$;
$R \leftarrow \text{sRGB}\big(\textsc{GamutFit}(L', \mathbf{c}')\big)$;
$\alpha \leftarrow W \cdot A$; $O \leftarrow I + \alpha(R - I)$ where $\alpha > 0$, else $O \leftarrow I$;
$\textsc{Verify}(I, O, W)$; **cache** (I, M, P, parts) under a new id;
Slider(id, T'): load (I, M, P, parts) and repeat lines 2 to 9 with T' (no VLM, no SAM, no diffusion);



Figure 9: Slider sweep on a real product photo: one selection, five colours chosen with the slider. Each re-render reuses the cached mask.

Table 3 compares the method with generative recolouring. The zero leak on people is partly by construction, because the same SAM mask defines the region and the score; it measures fidelity given the region. The hybrid row uses the region that Qwen chose to change and our transfer for the colour, which keeps Qwen's semantic choices (real stripes, a shoe sole as "trim") without its side effects.

6 Further pipelines

6.1 Pose transfer through words

Given both photos, the image model copies the reference instead of re-posing the person. We therefore never show it the reference (Fig. 12). OpenPose extracts 18 joints $\mathbf{k} \in \mathbb{R}^{18\times 3}$ from the reference; they are rescaled to the photo's frame. Qwen3-VL describes the overall pose ("sitting cross-legged on the floor"), and the joints are turned into precise words: for each limb segment (a, b) the direction of $\mathbf{k}_b - \mathbf{k}_a$ is mapped to words ("right forearm points up and to the left") and elbow and knee bends to angles. The edit uses only these words. After each round the pose is detected again and compared with the target using

$$\text{PCK@0.2} = \frac{1}{|V|} \sum_{j \in V} \mathbb{1}\big[\|\hat{\mathbf{k}}_j - \mathbf{k}_j\| < 0.2h\big], \tag{14}$$

with V the joints visible in both and h the size of the target skeleton. Limbs that are off by more than 30 degrees become a correction instruction for the next round (at most two corrections), and the best round in which Qwen3-VL confirms the same person is kept. This reached PCK 0.59 with the same person in 10 of 10 pairs, against 0.36 and 2 of 10 for Leffa.

6.2 Relighting with light transfer

A generative relight changes the light well but redraws texture and faces. We keep the new light and the old detail. In linear RGB, light is the smooth part of an image and detail the sharp part. With a Gaussian G_s of



Figure 10: Part-level programs on the same bag: an area-true upper half, a lower third, four stripes with crisp edges, the border, and a set difference (left half minus a horizontal band).



Figure 11: Inside the recolour (navy). From left: region weight W, colour affinity A (dark on the zip and edges), blend weight α, and the absolute change, which is exactly zero outside the bag. Right: OKLab lightness before and after inside the bag; the shading is kept.

width $s = 0.03\times$ the image diagonal, per channel c,

$$g_c = \mathrm{clip}\left(\frac{G_s(Q_c)}{G_s(P_c)}, \tfrac{1}{6}, 6\right), \qquad \mathrm{out}_c = P_c \cdot \tilde{g}_c, \tag{15}$$

where P is the photo, Q the model's relit version and $\tilde{g}$ the gain after an edge-aware guided filter (guide: the photo's grey image), which stops the gain from bleeding across object borders. In our test this kept 83 to 90% of the fine detail (against 29 to 34% for the model alone) with about 85% of the lighting effect.

IC-Light engine. As a second engine, the relight tool can use IC-Light (the foreground-conditioned Stable Diffusion 1.5 variant), which runs in its own small container. The subject is relit at 768 px with 25 DPM-Solver++ steps and a guidance scale of 2, from a light described in words (seven presets such as "light from the left", "golden hour" or "neon", or the user's own words, which pass the request guard first). The same detail transfer as above then puts the photo's full-resolution texture back under the new light.

6.3 Depth-aware portrait bokeh

Depth Anything V2 Small gives a relative depth $d \in [0,1]$ (1 = near), normalised by its 1st and 99th percentiles. The focus depth f is the median depth of the subject (found by Qwen3-VL and SAM, or clicked). With strength $\beta \in [0,1]$ and the image diagonal Δ, the blur radius is

$$\rho(p) = \beta \cdot 0.022\,\Delta \cdot \mathrm{clip}\left(\frac{|d(p) - f|}{0.45}, 0, 1\right), \qquad \rho = 0 \text{ on the subject.} \tag{16}$$

Before blurring, the subject is painted out of the background layer so that it cannot leave a halo. Blurring happens in linear light with a highlight weight $w = 1 + 6\,\mathrm{clip}\left((\bar{L} - 0.7)/0.3, 0, 1\right)$, so that bright points become round discs as with a real lens, while a normalised average keeps the overall level:

$$B_k = \frac{K_{\rho_k} * (wL)}{K_{\rho_k} * w}, \qquad k = 0, \ldots, 6, \tag{17}$$

Table 3: Recolour benchmark: 48 tasks (30 generated product photos, 18 people from RORD). Leak is the CIEDE2000 change on protected pixels of the object and of the background; coverage is the share of the requested part that changed colour.

Method	Leak object ↓	Leak background ↓	Hue error ↓	Coverage ↑	Shading corr. ↑	Time
Ours (recolour)	**0.00**	**0.00**	**0.6°**	**0.99**	**0.997**	3 s
Hybrid (Qwen decides where)	1.04	0.00	3.5°	0.82	0.870	21 s
Qwen-Image 2.1 edit	3.69	2.03	7.7°	0.83	0.735	21 s
InstructPix2Pix	20.10	12.76	12.0°	0.89	0.822	1 s

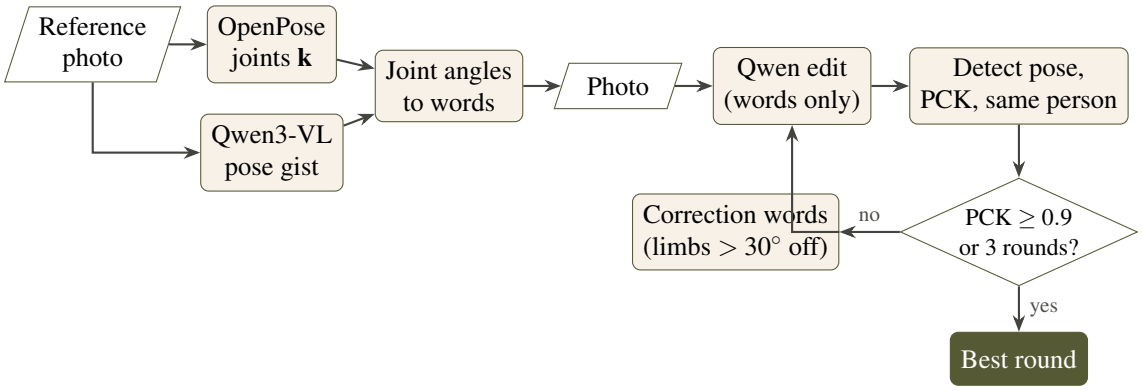


Figure 12: Pose transfer. The reference photo never reaches the image model; only words do.

with K_ρ an anti-aliased disc kernel and seven radii ρ_k evenly spaced up to the maximum. Each pixel interpolates between the two levels that bracket $\rho(p)$, and the sharp subject is composited back with a soft edge. Depth is cached, so the strength slider and click-to-refocus re-render on the CPU in about a second.

6.4 Dolly video from one photo

The server separates the photo into a subject cut-out (with soft alpha), a background plate in which the subject has been removed by the smart removal pipeline, and a depth map of the plate. The browser renders the camera move with WebGL (Fig. 14): the plate becomes a relief mesh with $z = (d - \frac{1}{2})\rho_z$ ($\rho_z = 0.32$), and the subject is a plane at depth $z_s = (d_s - \frac{1}{2})\rho_z + 0.02$, scaled by $k = (D - z_s)/D$ so that it lines up with the photo at the start, where $D = \frac{1}{2}/\tan(\phi_0/2)$ is the camera distance for a field of view $\phi_0 = 38°$. With $e(t) = \frac{1}{2} - \frac{1}{2}\cos(2\pi t)$ over a 4 s loop, the presets are:

- push in: $z_c = D(1 - 0.24\,e)$;
- dolly zoom: $z_c = D(1 + 0.7\,e)$ with $\phi(t) = 2\arctan\big(\tan(\phi_0/2)\,(D - z_s)/(z_c - z_s)\big)$, which keeps the subject's size constant while the background stretches;
- orbit and rise: small circular and vertical camera paths looking at the subject.

The browser records one loop as a video file, so no GPU is used per frame.

6.5 Glow light

The user clicks where a light should be. Depth Anything V2 gives disparity d (1 = near), which becomes a surface $z = 1 - d$ with normals from its image gradients,

$$\mathbf{n} = \frac{(-\frac{1}{4}W\,\partial_x z,\ -\frac{1}{4}H\,\partial_y z,\ -1)}{\|\cdot\|}. \tag{18}$$

For a light at image position (p_x, p_y) and height z_ℓ, each pixel gets the vector $\mathbf{v} = (p_x - x,\ p_y - y,\ -(z(x,y) - (1 - z_\ell) + 0.05))$ with length r, and its planar distance ρ. With size σ and softness κ, the falloff and the added



Figure 13: Bokeh render. Background lights turn into discs and the subject keeps its fur detail. (Rendered with an illustrative depth map; in the product the depth comes from Depth Anything V2.)

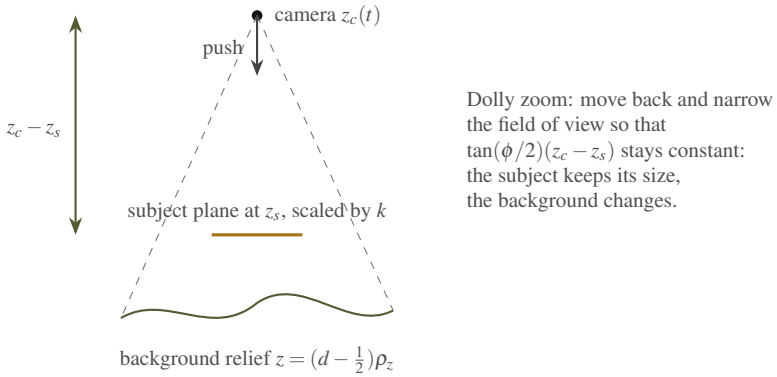


Figure 14: Geometry of the dolly video (side view).

light in linear RGB are

$$e = \mathrm{clip}\Big(\frac{\sigma-\rho}{\sigma\kappa}, 0, 1\Big), \qquad f = \frac{e^2(3-2e)}{1+(r/\sigma)^2}, \qquad C = \frac{I}{100}\,4f\,\big(0.15+0.85\max(\mathbf{n}\cdot\hat{\mathbf{v}},0)\big)\,\mathbf{c}, \tag{19}$$

where I is the intensity and $\mathbf{c}$ the light colour. The team's model adds this light, $L' = L + C\,(0.25 + 0.75L)$. On a dark photo that paints a coloured haze over the scene instead of showing it: the light cannot reveal a portrait whose pixels it never multiplies. LumenAI therefore lights by *exposure*. A real light multiplies the reflectance that is already recorded, so with the normalised shape $s \in [0,1]$ (the falloff without the inverse-square term) and the tint $\mathbf{t} = \mathbf{c}/\max(\mathbf{c})$,

$$F = \Big(\frac{I}{65}\Big)^{0.6} s\,\big(0.35+0.65\max(\mathbf{n}\cdot\hat{\mathbf{v}},0)\big), \qquad L' = \tilde{L}\,\big(1+(g^F-1)\,\mathbf{t}\big)+0.03\,F\,\mathbf{c}, \qquad g = \mathrm{clip}\Big(\frac{0.22}{Y_{60}}, 1.6, 256\Big), \tag{20}$$

where Y_{60} is the 60th percentile of the luminance inside the light and g is an automatic exposure: at the default strength the centre of the light reaches mid-grey however dark the photo is, and a bright photo gets only a gentle lift. Because a gain of up to 256 also amplifies JPEG noise, $\tilde{L}$ blends the photo towards an edge-preserving (bilateral) version of itself in proportion to the gain, and values above 0.8 pass through a soft shoulder instead of clipping. Figure 15 shows the difference on a nearly black photo. A spot mode multiplies the shape by a smooth cone weight around a chosen direction. The depth map is cached, so moving the light or changing its colour re-renders on the CPU. After the first click the editor also receives the photo, its denoised copy and the depth map at up to 1024 px, and a WebGL fragment shader evaluates the same equations under the cursor in real time, so the light can be swept across the scene as a live preview before it is placed.



Figure 15: Glow light on a nearly black photo. From the left: the photo, the additive light (a haze that hides the wall), and LumenAI's exposure light in white and warm colours, which reveals the portrait and the writing in their own colours.

6.6 Auto collage

From a set of photos (optionally with a theme such as "summer trip"), each photo gets a technical score t (sharpness, exposure, clipped pixels), an aesthetic score a from CLIP (the preference between a positive and a negative aesthetic prompt, $\sigma(s^+ - s^-)$) and a relevance r to the theme (CLIP cosine similarity). After robust normalisation the ranking score is

$$S = 0.46\,a + 0.29\,t + 0.25\,r \quad \text{(with a theme)}, \qquad S = 0.62\,a + 0.38\,t \quad \text{(without)}. \tag{21}$$

Near-duplicates (by CLIP embedding and a perceptual hash) are dropped, faces or a saliency centroid choose each crop, and three layout designs are rendered deterministically for the user to pick from.

6.7 Other tools

- **Expand**: the photo is placed on a larger canvas and Qwen-Image with the community Outpaint LoRA paints the new borders in 10 steps (LPIPS 0.364 on restored borders, against 0.505 for our earlier method). The user picks a preset ratio, types any ratio between 1:4 and 4:1, or grows every side by 5 to 100%; the shorter side grows to reach the ratio and the canvas is capped at 1536 px with sides that are multiples of 16.
- **Adding objects** is done by prompt ("add a lamp on the table"). An earlier dedicated tool, in which Qwen3-VL proposed objects and sizes and the model filled a greyed-out box, succeeded on 27 of 32 placements; it was removed from the toolbar because a prompt does the same job.
- **Suggestions**: Qwen3-VL looks at the photo and proposes about six edits made for it (light and colour, background, framing, a distraction to remove, one fitting detail, an artistic look), each as a ready-to-run instruction that goes through the normal edit route and the request guard.
- **Move**: the selection is lifted, the hole is filled by smart removal, and the object is placed at the new position and scale with a feathered alpha.
- **Product shot**: FLUX.2 klein shows the product from four angles (45 degrees left, side, back and top, one seed each), following the team's klein pipeline; the four views appear as variations. The earlier scene mode (Qwen-Image places the product in a scene, DINOv2 product similarity 0.90 against 0.82 for klein) stays in the API but is no longer in the toolbar.
- **Style**: following the team's style pipeline, 27 classic artworks or the user's own reference photo are applied with Magenta arbitrary image stylisation (TensorFlow Hub), a feed-forward network that takes the photo and the artwork (at 256 px) and returns the stylised photo in 0.04 s with 1.3 GB of GPU memory. A strength slider blends the result with the photo. FLUX.2 klein was evaluated as an



(a) "Make it navy blue"



(b) "A highly aesthetic view of the New York City skyline"



(c) A re-posed person (left) turned into a 3-D model, four views (earlier Hunyuan3D-2 engine)

Figure 16: Product examples: recolouring, text-to-image generation, pose and 3-D.

alternative (Table 8); it keeps more of the photo's content but is about 80 times slower and needs a 13 GB container, and Magenta's stronger painterly texture was preferred for this tool.

- **Reference images**: up to ten reference photos are shown as attachments named "image 1" to "image 10". The edited photo is the model's `<image1>`, so the server rewrites "image k" in the request to `<image`$k+1$`>`; references are resized to at most 768 px to bound memory.
- **Upscale**: TADSR, a one-step diffusion super-resolution model, at $\times 4$; $\times 2$ is the $\times 4$ result resized.
- **View in 3D**: the object is cut out, Qwen-Image completes hidden or cut-off parts on a white background, and a single-image 3-D model builds a UV-textured mesh, shown in the browser and downloadable as GLB. Two engines are offered. Stable Fast 3D (SF3D) predicts the mesh in one forward pass (11.0 GB, 34 s on a T4) but has to guess depth and the unseen back from one view, so objects photographed head-on tend to come out thin. SPAR3D, the default, first samples a point cloud of the whole object with a small diffusion model and then builds the mesh from it, which gives fuller depth and back sides. Both replaced Hunyuan3D-2 (11.5 GB, 54 s). The user can turn the model, capture the current view and *use this view*: the old object is removed by smart removal and the rendered view is scaled to the old object's box on the same ground line. The placed object can then be rotated (any angle) and resized in the browser before it is accepted; the server returns the clean plate and the cut-out view, so each change is a local redraw with no GPU work.
- **Watermark**: exports carry an invisible 32-bit mark ("LUMN") embedded with DWT, DCT and SVD; a verify endpoint reads it back.

6.8 The final pipeline at a glance

Table 4 lists every tool of the final system with the pipeline and models behind it.

Table 4: The final pipeline: each tool, how it works, and where it runs (M = main container, S = side container, B = browser, C = CPU in the main container).

Tool	Pipeline	Models	Where
Select, brush	clicks, box or stroke to a mask; brush can snap to the object	SAM 2.1	M
Edit with words	guard, router, instruction edit (local or global), up to 10 references	tier: klein or Qwen-Image	S / M
New image	text to image at a chosen ratio	tier: klein or Qwen-Image	S / M
Remove	mask (or grounded by Qwen3-VL), shadow extension, removal	Qwen-Image, SAM 2.1	M
Move	lift, remove, place with feathered alpha	Qwen-Image, SAM 2.1	M
Recolour	word program, mask algebra, OKLab transfer, slider	Qwen3-VL, SAM 2.1	M + C
Expand	preset, custom ratio or grow, outpaint	Qwen-Image + Outpaint LoRA	M
Product shot	four angles of the product	klein	S
Style	27 artworks or own reference	Magenta (TF)	S
Pose	joints from the reference, words, edit	OpenPose, Qwen3-VL, Qwen-Image	M
Relight	generative relight plus detail transfer	Qwen-Image or IC-Light	M / S
Glow light	depth surface, Lambert light, linear blend	Depth Anything V2	M + C
Bokeh	depth layers, disc blur in linear light	Depth Anything V2, SAM 2.1	M + C
Dolly video	subject, plate, depth; WebGL camera	Depth Anything V2, Qwen-Image	M + B
Collage, auto collage	templates; CLIP ranking and three designs	CLIP ViT-B/32	C
Upscale	one-step diffusion super-resolution	TADSR	S
View in 3D	cut-out, completion, mesh, view capture, replace, rotate	Qwen-Image, SPAR3D or SF3D	M + S + B
Suggestions	photo-specific edit ideas, run as edits	Qwen3-VL	M
Export	invisible 32-bit watermark	DWT-DCT-SVD	C

7 Systems engineering

- **4-bit weights.** The Qwen-Image transformer and its Qwen3-VL text encoder load in nf4 (bitsandbytes).
- **Encoder offloading.** The 4.4 GB text encoder lives on the CPU and moves to the GPU only to encode a prompt or answer a question. The output is bit-identical and the peak drops from 20.7 to 14.0 GB at 1024 px.
- **Prompt-embedding cache.** The last eight encoder outputs are kept, keyed by the prompt and a hash of the exact input pictures. Repeats (“Another version”, retries) skip the encoder trip.
- **VAE tiling off.** Tiling saved about 0.9 GB but changed the picture the model conditions on and made it keep the person in 14 of 50 removals, so it is disabled.
- **Lumina in 4-bit.** FLUX.2 klein’s 8 GB Qwen3 text encoder in nf4 brings the container from 18.6 to 13.35 GB at the same 1.1 s per image; CPU offload would reach 9.5 GB but takes 7.5 s (Table 6 and Fig. 17).
- **TF32** matrix multiplications, **CPU-only re-renders** for the recolour and bokeh sliders, and **watermarking only on export** (it used to cost 1 to 2 s per preview).
- **Serverless scaling.** Containers start on the first request (1 to 3 minutes from the weight cache) and stop after 10 idle minutes.

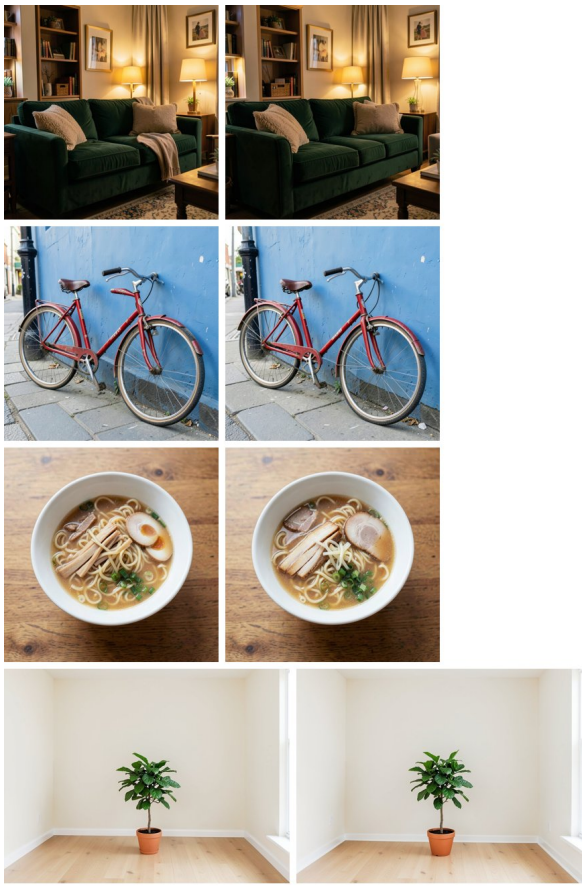


Figure 17: Lumen 1.1 Lumina with a full-precision (left) and a 4-bit (right) text encoder, same seeds. The 4-bit images differ in composition but not in quality, at 5.3 GB less memory. The bottom row is an edit ("add a green potted plant").

8 Benchmarks

All numbers were measured on one NVIDIA L40S after a warm-up run (the deployed system now runs the main container and Lumina on an H100, so these times are conservative). Peak memory is the peak GPU memory allocated while the model ran. Where a judge was needed, Qwen3-VL and Gemini scored outputs from 0 to 10; all other metrics are computed directly from pixels.

8.1 Speed and memory

Table 5: Peak GPU memory and execution time of the shipped models (1024 px). *SPAR3D (L4) and SF3D (T4) were measured separately on the same photo (first request, including background removal): SPAR3D 7 584 faces and a 0.4 MB GLB, SF3D 17 612 faces and 0.7 MB. IC-Light was added after this run.

Feature	Model	Peak VRAM	Time
Create an image	Lumen 1.1 Lumina (FLUX.2 klein 4B, 4-bit encoder)	13.4 GB	1.1 s
Create an image	Lumen 1.3 Lux (Qwen-Image 2.1, 40 steps)	12.5 GB	20.6 s
Edit with words	Lumen 1.1 Lumina	13.4 GB	1.7 s
Edit with words	Lumen 1.3 Lux	13.8 GB	35 s
Product shot	Qwen-Image 2.1	14.2 GB	23.5 s
AI upscale $\times 4$	TADSR	5.7 GB	$<$0.5 s
View in 3D (default)	SPAR3D (L4)*	11.0 GB	29 s
View in 3D (fast option)	Stable Fast 3D (T4)*	11.0 GB	34 s
View in 3D (earlier engine)	Hunyuan3D-2 (L40S)	11.5 GB	54 s
Safety check	Qwen3-VL (shared) + DeBERTa-v3	+0.8 GB	2.3 s
Invisible watermark	DWT-DCT-SVD	CPU	0.08 s

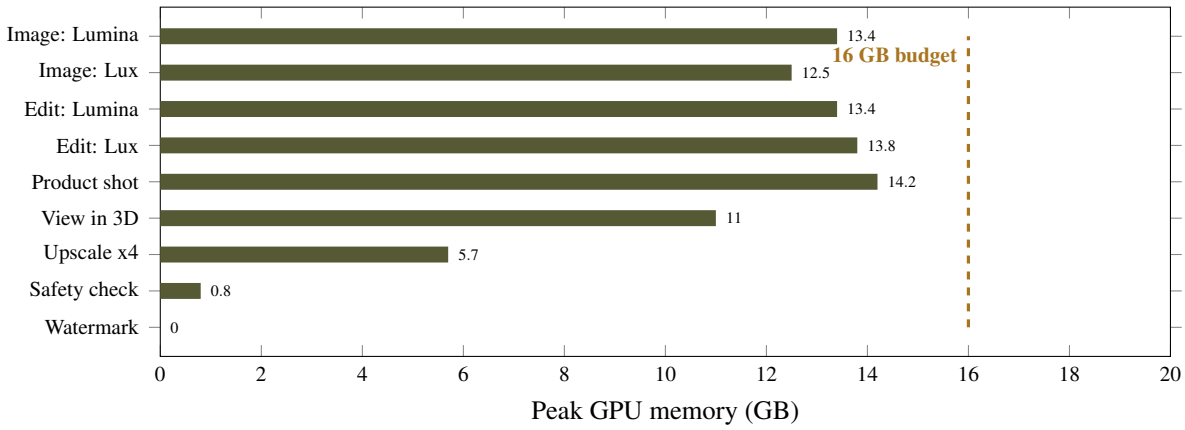


Figure 18: Every shipped model stays under the 16 GB budget.

Table 6: Ways to load Lumen 1.1 Lumina (3 prompts and one edit at 1024 px).

Mode	Peak VRAM	s / image	Edit	Note
bf16, all on GPU	18.6 GB	1.11	1.73	over budget
4-bit text encoder (shipped)	**13.35 GB**	**1.13**	**1.72**	same quality by eye
model CPU offload	9.5 GB	7.55	7.42	7$\times$ slower

8.2 Quality

Table 7: Quality against the alternatives we evaluated. Arrows show which direction is better.

Task (data)	Metric	LumenAI	Alternatives
Object removal (RORD)	LPIPS in the hole ↓	**0.108**	LaMa 0.151, MAT 0.179, Segformer+LaMa 0.149
Pose transfer (10 pairs)	PCK@0.2 ↑ / same person	**0.59 / 10 of 10**	Leffa 0.36 / 2 of 10
Upscale ×4 (16 photos)	LPIPS ↓	**0.217** (TADSR)	Lanczos 0.303, Qwen refine 0.290
Product shot (24 cases)	DINOv2 product similarity ↑	**0.900**	FLUX.2 klein 0.815
Text to image (20 prompts)	judge: prompt / quality ↑	9.7 / 9.8 (Lux)	klein 9.7 / 9.7, Z-Image 9.6 / 9.6
Instruction edits (GEdit, 33)	overall ↑	7.36 (Qwen), **7.80** (Lumina)	InstructPix2Pix 2.91
Style transfer (48 pairs)	content kept (DINOv2) ↑ / style (Gram, log) ↓	**0.81** / 1.25 (klein)	Magenta (TensorFlow) 0.44 / **0.87**
Recolour (48 tasks)	leak ΔE_{00} ↓ / shading ↑	**0.00 / 0.997**	Qwen edit 3.69 / 0.735
Motion deblur (16 photos)	LPIPS ↓	**0.142**	blurred 0.248, Wiener 0.331
Watermark after JPEG 75	bit accuracy ↑	**0.99** (DWT-DCT-SVD)	DWT-DCT 0.53
Safety (60 requests)	harmful / injections blocked, false blocks	19/20, 10/10, 0	SingGuard-4B 19/20, 8/10, 0 (12.6 s, 9.1 GB)

Style transfer. We compared FLUX.2 klein (85 % strength) with Magenta arbitrary image stylisation from TensorFlow Hub, the classic feed-forward method, on 12 photos and 4 artworks (Table 8 and Fig. 19). The two methods sit at opposite ends of the same trade-off. Magenta copies the artwork's texture statistics most closely (lower Gram loss, and a higher CLIP similarity to the artwork) and is very cheap, but it breaks the photo: its DINOv2 content similarity is 0.44, and faces and products dissolve into texture. Klein keeps the content (0.81, better on 46 of 48 pairs) and repaints the scene with the artwork's palette and light, at the cost of a weaker texture match and about 3 s per image. The editor uses Magenta: the tool is meant for strong painterly looks, and Magenta gives them in 0.04 s; users who need the subject kept exactly can lower the strength slider.

Table 8: Style transfer on 48 photo and artwork pairs (mean, standard deviation in brackets). Content scores compare the result with the photo, style scores with the artwork. Times are warm, on an L40S.

Method	DINOv2 content ↑	CLIP content ↑	CLIP style ↑	Gram loss ($\log_{10}$) ↓	Time
Magenta (TensorFlow Hub)	0.44 (0.20)	0.64 (0.07)	**0.62** (0.07)	**0.87** (0.44)	**0.04 s**
FLUX.2 klein, raw	0.80 (0.13)	0.70 (0.08)	0.60 (0.09)	1.39 (0.43)	3.1 s
FLUX.2 klein, 85 % blend	**0.81** (0.12)	**0.71** (0.08)	0.60 (0.09)	1.25 (0.42)	3.1 s

Peak GPU memory: Magenta 1.3 GB, klein 13.35 GB (its container). Klein's time includes the call between containers. Klein (85 %) beats Magenta on DINOv2 content in 46 of 48 pairs and on CLIP content in 39; Magenta wins on CLIP style in 33 and on Gram loss in 39.

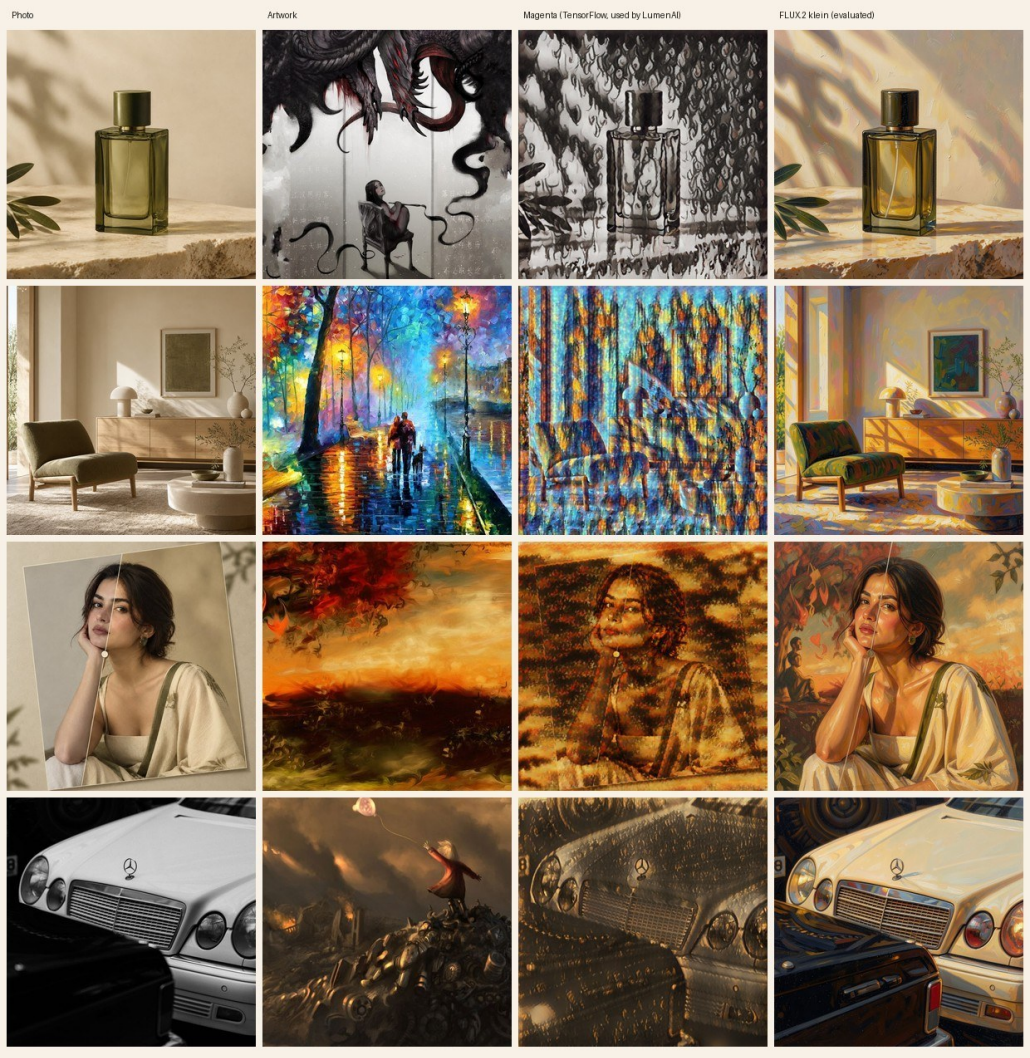


Figure 19: Style transfer examples. Magenta (third column) matches the artwork's texture but loses the product, the room and the face; klein (right) keeps them and takes the artwork's palette and light. Klein can also borrow scene elements from the artwork (the figures behind the portrait), which the instruction tries to prevent.

Decisions taken from the benchmark. FLUX.2 klein replaced our model for the fastest tier because it matched the quality at a twentieth of the time. Qwen-Image stayed for product shots because klein redraws logos. Our removal, pose, recolour and guard pipelines stayed because they beat the alternatives. TADSR and the DWT-DCT-SVD watermark were adopted. After the benchmark, SF3D replaced Hunyuan3D-2 for View in 3D (it was gated during our run; the team's tests showed cleaner meshes; on a T4 it needs 11.0 GB and 34 s, against 11.5 GB and 54 s on an L40S for Hunyuan3D-2); because single-pass SF3D made head-on objects thin, SPAR3D was then added as the default 3-D engine, FLUX.2 klein took on product angles, where redrawing is acceptable, and Magenta took on artwork styles. Qwen-Image-Edit-2511 with a multiple-angles LoRA produced the best new camera angles but needed 42.5 GB and 140 s per image, so it was not adopted.

9 Limitations

- Qwen-Image edits take tens of seconds end to end; Lumina is interactive only while its container is warm.
- Geometric recolour parts (halves, stripes) cut across materials on people; clothing needs the `part` operator. Stripes are flat in the image plane unless a depth map is supplied.
- The dolly video is a 2.5-D effect: large camera moves reveal the edges of the background plate.

- Several quality scores come from a vision-language judge, which can be lenient; pixel metrics are reported where ground truth exists.
- Cold starts take 1 to 3 minutes, and some side models carry academic or community licences (TADSR, SF3D, SPAR3D).
- IC-Light has not yet been through the full benchmark, and SPAR3D and SF3D were each timed on a single request.
- SPAR3D can attach a thin sliver of the ground (snow under the fox in our test) to the mesh.

10 Conclusion

LumenAI shows that a single text box can drive a broad editor when the system routes each request to a pipeline built for it, and that careful systems work keeps a multi-model editor inside 16 GB per model. The recolour slider illustrates the general idea: let a language model decide what the user means, and let exact mathematics change the pixels. The result is an edit that is faster, more faithful and fully protected outside the selection.

A API endpoints

Endpoint	Purpose
`/edit`	routed edit from a prompt (with optional mask and reference)
`/generate`	text to image with the chosen tier
`/segment`	SAM 2.1 mask from points or a box
`/recolour`, `/recolour-shade`	exact recolour and slider re-render
`/expand`, `/suggest`, `/move`	outpainting, photo-specific suggestions, moving
`/pose`, `/style`, `/relight`, `/product`	pose, style, light and product tools
`/bokeh`, `/bokeh-adjust`, `/layers`	depth tools (bokeh, dolly layers)
`/cutout`, `/upscale-ai`, `/view3d`, `/replace-object`	background removal, upscaling, 3-D, placing a captured view
`/glow`, `/glow-adjust`	glow light and its re-render
`/auto-collage`, `/product-angles`	ranked collage designs, four product views
`/export`, `/verify`, `/health`	watermarked export, mark check, status

B Reproducibility

The code is organised as a frontend (static HTML and JavaScript), a Python backend and deployment files for a serverless GPU platform. The offline test suite (135 tests) runs without a GPU using stand-in models, and a demo server runs the full API with placeholder results so that the editor can be tried without a GPU. The benchmark runner regenerates every number in Section 8.